

GENETIC ALGORITHMS FOR MINIMIZING TARDINESS ON PARALLEL MACHINES WITH SETUPS

Vinicius Amaral Armentano and Renata Mazzini

Faculdade de Engenharia Elétrica e de Computação - FEEC
Universidade Estadual de Campinas - UNICAMP
C.P. 6101 - CEP 13083-970 - Campinas - S.P. - Brazil
Phone: :5-19-788-2143
Fax 5-19-788-1395
e-mail: vinicius@densis.fee.unicamp.br, mazzini@densis.fee.unicamp.br

Abstract: In this paper a genetic algorithm for the identical parallel machine scheduling problem with sequence dependent setup times and due dates is presented. The algorithm is described and computational experiments are reported. The test problems ranged from 30 jobs and 2 machines to 110 jobs and 6 machines. The algorithm presents a good performance and the computational time spent indicates that even larger problems can be solved without loss of efficiency.

Copyright © 1998 IFAC

Keywords: Genetic Algorithms, Parallel processors, Scheduling algorithms, Heuristics, Optimization problems.

1. INTRODUCTION

This paper deals with the problem of scheduling n jobs on m identical parallel machines in order to minimize total tardiness. Each job J_j has a processing time p_j and a due date d_j . If two jobs J_i and J_j are processed consecutively on the same machine there is a setup time S_{ij} between the completion time of J_i and the starting time of J_j . When a job J_j is the first to be processed on any machine an initial setup time S_{0j} is incurred and a fictitious job J_0 is included in the problem to represent this initial setup time. This problem is called PST from the keywords Parallel machines, Setup times and Tardiness.

In order to formulate the problem let $x_{ij}^k = 1$ if J_j follows immediately J_i on machine k and $x_{ij}^k = 0$, otherwise. Define C_j as the completion time of J_j and T_j its tardiness given by $\max\{0, C_j - d_j\}$. The optimization model for the PST is similar to that presented by Guinet (1991) for the makespan criterion.

$$\text{Minimize } T = \sum_{j=1}^n T_j \quad (1)$$

subject to

$$\sum_{k=1}^m \sum_{i=0, i \neq j}^n x_{ij}^k = 1 \quad j=1, \dots, n \quad (2)$$

$$\sum_{j=1}^n x_{0j}^k \leq 1 \quad k=1, \dots, m \quad (3)$$

$$\sum_{i=0, i \neq h}^n x_{ih}^k - \sum_{j=0, j \neq h}^n x_{hj}^k = 0 \quad k=1, \dots, m; h=1, \dots, n \quad (4)$$

$$C_j \geq C_i - M + (S_{ij} + p_j + M) \cdot \sum_{k=1}^m x_{ij}^k \quad (5)$$

$$i=0, \dots, n; j=1, \dots, n$$

$$T_j \geq C_j - d_j \quad j=1, \dots, n \quad (6)$$

$$C_j \geq 0; T_j \geq 0 \quad j=1, \dots, n \quad (7)$$

$$x_{ij}^k \in \{0, 1\} \quad (8)$$

$$i=0, \dots, n; j=1, \dots, n; k=1, \dots, m$$

In this formulation, M is a large number. Constraints (2) ensure that each job is processed only once. Constraints (3) ensure that each machine has at most

one first job, i.e., one job sequence. Constraints (4) specify that each job must have a predecessor job and a successor job. Constraints (5) prevent the processing for two or more jobs simultaneously. Moreover, these constraints also prevent a job to be the predecessor and successor of the same job. Constraint (6) and $T_j \geq 0$ define the tardiness.

This problem is NP-hard since minimizing total tardiness in one machine without setup times is NP-hard (Du and Leung, 1990). For this reason a heuristic method was developed to obtain good solutions for large problems in a reasonable amount of time.

Good surveys on parallel machine scheduling problems can be found in Cheng and Sin (1990) and in Kawaguchi and Kyan (1988). In both works the importance of considering aspects such as sequence dependent setup times and due dates in scheduling problems is highlighted. However, the number of papers that deal with parallel machine scheduling with such aspects is very small (Guinet, 1991 and 1995). The research with parallel machines subject to sequence dependent setup times is mainly concentrated on the minimization of makespan (Guinet, 1993; França *et al*, 1995), average flowtime and time spent on setups (Rajgopal and Bidanda, 1991).

Bernardo and Lin (1994) consider the bi-criteria scheduling problem resulting from minimizing total tardiness and setup costs. Unfortunately, setup costs can not be directly translated into setup times. The work presented by Lee and Kim (1993) consists of the development of a neural network to compute two parameters for a dispatching rule proposed by Lee and Pinedo (1992). This rule could be used to evaluate the performance of the algorithm proposed here but the parameter values are not specified in the paper.

The purpose of this study is to evaluate the potential of genetic algorithms for solving the PST. The genetic algorithm scheme proposed is presented in section 2. The exogenous parameters of the algorithm were determined by statistical methods and its performance was evaluated against upper bounds yielded by a simple constructive heuristic. This heuristic and the generation of test problems are described in section 3 Computational results are presented in section 4 and concluding remarks in section 5.

2. A GENETIC ALGORITHM FOR THE PST

Genetic Algorithms were proposed by John Holland (Holland, 1992) and can be used as a technique for solving combinatorial optimization problems. Good surveys of the method can be found in Goldberg

(1989) and Michalewicz (1992).

The genetic algorithm presented in this paper is based on the general scheme described by Goldberg (1989). Such a scheme was chosen due to its simplicity and its similarity with the traditional version of the algorithm. Figure 1 presents an outline of the implemented algorithm.

Each population P_t ($t = 0, \dots, t_{\max}$) consists of $i_{\max}$ feasible solutions of the PST problem. These solutions are represented by strings of $(m + n)$ elements which store the permutation of jobs and machines, starting with machine M_1 . The machine indices have a negative sign to distinguish them from the jobs. For example, for a problem with $m = 2$ and $n = 5$ the string $(-1 \ 1 \ 2 \ -2 \ 3 \ 4 \ 5)$ represents a solution in which machine M_1 processes jobs J_1 and J_2 in that order and machine M_2 processes jobs J_3 , J_4 and J_5 in that order.

Given m and n there are $(m + n - 1)!$ different permutations of jobs and machines since machine M_1 is fixed in the first position. This number of permutations is greater than the number of feasible solutions for the PST since the representation has some redundancies. For example, the strings:

$(-1 \ 1 \ 2 \ 3 \ 4 \ 5 \ -2 \ -3)$ and $(-1 \ 1 \ 2 \ 3 \ 4 \ 5 \ -3 \ -2)$

represent the same solution in which machine M_1 processes all jobs and machines M_2 and M_3 are idle.

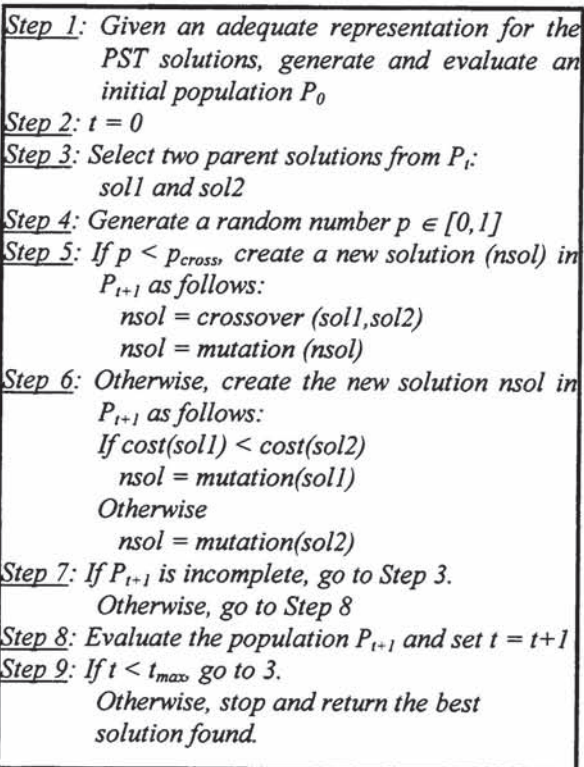


Figure 1 - Scheme of the implemented algorithm.

The initial population P_0 is randomly generated and evaluated according to the tardiness of its $i_{\max}$

solutions. The next step corresponds to creating a new population P_{t+1} from P_t . For this, $i_{\max}$ new solutions are created from the current solutions through a loop in which two strings in P_t are selected to generate each string in P_{t+1} . The first genetic operator in this loop selects parent strings from P_t and the selection is carried out by means of a roulette wheel constructed as follows. Let F be the tardiness sum of the solutions S_i in P_t , $i = 1, \dots, i_{\max}$.

$$F = \sum_{i=1}^{i_{\max}} T(S_i) \quad (9)$$

For a maximization problem, the selection probability of each solution is calculated as:

$$p(S_i) = \frac{T(S_i)}{F}, \quad i = 1, \dots, i_{\max} \quad (10)$$

Since PST is a minimization problem, the best solutions have the smaller tardiness. For this reason, expression (10) is inadequate since it assigns higher probabilities to poor solutions. A suitable way for assigning higher probabilities to good solutions is as follows:

$$p(S_i) = \left[1 - \frac{T(S_i)}{F} \right] \cdot \frac{1}{i_{\max} - 1}; \quad i = 1, \dots, i_{\max} \quad (11)$$

where the factor $1/(i_{\max} - 1)$ ensures that the sum of probabilities over the entire population is 1.

From the selection probabilities (11), cumulative values can be computed:

$$q(S_i) = \sum_{j=1}^i p(S_j); \quad i = 1, \dots, i_{\max} \quad (12)$$

In order to select a string using this roulette wheel, a random number $r \in [0,1]$ is generated. If $r \leq q(S_1)$, the string S_1 is selected. Otherwise, a string S_i such that $q(S_{i-1}) < r \leq q(S_i)$, $i = 2, \dots, i_{\max}$ is selected.

The next genetic operator described is the crossover. For the PST an adaptation of the Edge Recombination (ER) operator is used. This operator was developed for the Traveling Salesman Problem and is presented in detail in Michalewicz (1992). The parameter p_{cross} controls the crossover utilization.

For each pair of parent solutions selected in Step 3, a random number $p \in [0,1]$ is generated. If this number is less than p_{cross} the parent solutions are recombined to create a descendant. For this, a list of successors for each job and machine in both parent solutions is constructed. One example of such a list is shown in Figure 2. The symbol "x" is used to indicate that a job (or machine) has no successor in sol1 or sol2.

In the example, the elements of the first column of the successor list represent the successors of machine M_3 and the second column, the successors of machine M_2 . Job J_2 is the first job processed on machine M_1 in sol1 and job J_6 has no successor either in sol1 or sol2.

All information needed to construct a descendant solution is in the successor list. Starting from M_1 , the successor of this machine is randomly chosen from its successors in sol1 and sol2. The best parent is more likely to pass its characteristics to the descendant solution by assigning to the successor coming from sol1 a probability

sol1 = (-1 2 3 -2 1 4 -3 5 6)	
sol2 = (-1 5 3 2 -2 -3 1 4 6)	
Successor list:	
	<u>-3 -2 -1 1 2 3 4 5 6</u>
in sol1	5 1 2 4 3 -2 -3 6 x
in sol2	1 -3 5 4 -2 2 6 3 x

Figure 2 - Example of the list of successors used by the crossover operator

$$p_1 = \frac{T(\text{sol2})}{T(\text{sol1}) + T(\text{sol2})} \quad (13)$$

and to the successor coming from sol2 a probability

$$p_2 = 1 - p_1 \quad (14)$$

For example, in the successor list of Figure 2, starting with M_1 , the probability of choosing J_2 is p_1 and the corresponding probability for J_5 is p_2 . Assuming that J_5 is chosen, the next step is the choice of its successor. This process is repeated until all jobs and machines have been chosen without repetition. In case of a deadlock, the next element of the descendant solution is randomly chosen from the unscheduled jobs or machines.

In step 5 of Figure 1 the descendant solution obtained by the crossover operator can be modified by mutation. The same is true for the best of the parent solutions in step 6 in case crossover is not used. In either case, let S_i be the solution to which mutation is applied. The control parameter of the mutation operator is p_{mut} and it is tested against a randomly generated number $r \in [0,1]$. When $r \geq p_{\text{mut}}$ the mutation operator has no effect and S_i remains unaltered. Otherwise two types of mutation can be applied to S_i : exchange (E) and insertion (I).

The first type of mutation consists of the exchange of two elements of the string S_i . These elements represent two jobs, a job and a machine or two machines. The second type of mutation alters the position of only one element of the string S_i by

inserting it in a position other than its current position. Again, this element can be a job or a machine. The choice of which type of mutation is applied to a string S_i depends on the expected proportion between the occurrences of exchanges and insertions. This proportion is denoted by $E:I$.

When the initial population P_0 is created, the best solution in this population is stored as S^* . Each time a new solution is created through crossover or modified by mutation, its tardiness is compared to $T(S^*)$ and if the new solution is better than S^* , it replaces S^* as the best solution found so far. When the algorithm stops in step 9, S^* is its final solution with total tardiness $T(S^*)$.

In the following section the generation of test problems is presented along with the heuristic used to obtain the upper bounds for these problems.

3. TEST PROBLEMS AND UPPER BOUNDS

In order to test the proposed algorithm 750 test problems were generated, 30 for each combination of the values $m \in \{2, 3, 4, 5, 6\}$ and $n \in \{30, 50, 70, 90, 110\}$. All problem parameters are integer numbers and they were randomly generated from a uniform distribution in the following intervals: processing times: $[1, 100]$, setup times: $[0, 20]$ and due dates: $[0, \alpha \cdot P]$, where P is the sum of the processing times and α is a parameter that determines the due date tightness. The values used for α were 0.8, 1.0 and 1.2. Then, for each combination of m and n , 10 problems were generated for each value of α .

Since the optimal solutions for these test problems are unknown and there is no strong lower bound for the PST, the solutions obtained by the genetic algorithm are evaluated against a simple constructive heuristic. Barr *et al.* (1995) recommend the use of upper bounds when there are no lower bound nor classical instances with known optimal solution.

Figure 3 shows the steps of the heuristic where the following notation is used:

- T_k : sum of tardiness of jobs processed on machine M_k ;
- x_k : number of jobs processed on machine M_k ;
- $\text{pos}(j, k)$: position of job J_j in the processing sequence of machine M_k ;
- $\Delta T(j, k)$: increase in T_k when an unscheduled job J_j is inserted in position $\text{pos}(j, k)$.

The performance of the genetic algorithm for the PST is measured by the gain with respect to the heuristic described. This gain is computed as:

$$\text{Gain} = \frac{T(H) - T(GA)}{T(H)} \cdot 100\% \quad (15)$$

where $T(H)$ is the tardiness value of the heuristic solution and $T(GA)$ is equal to $T(S^*)$ given by the genetic algorithm.

Step 1:	Set $x_k = T_k = 0$, $k = 1, \dots, m$.
Step 2:	For $k = 1, \dots, m$ do: Take the unscheduled job with the earliest due date and assign it to machine M_k . Set $x_k = 1$ and compute T_k .
Step 3:	For each unscheduled job J_j and for each machine M_k , $k = 1, \dots, m$: Find the position $\text{pos}(j, k)$ where the insertion of J_j causes the least increase $\Delta T(j, k)$, $1 \leq \text{pos}(j, k) \leq x_k + 1$.
Step 4:	Find the best value of $\Delta T(j, k)$ to all unscheduled jobs J_j and all machines M_k , $k = 1, \dots, m$. Let j^* and k^* be the job and machine for which $\Delta T(j^*, k^*) = \min\{\Delta T(j, k), \forall j, \forall k\}$.
Step 5:	Insert job J_{j^*} in position $\text{pos}(j^*, k^*)$ on machine M_{k^*} , set $x_{k^*} = x_{k^*} + 1$ and compute T_{k^*} .
Step 6:	If all jobs are scheduled, go to Step 7. Otherwise, go to Step 3.
Step 7:	Compute the total tardiness of the complete solution. Stop.

Figure 3 - Description of the heuristic

The computational experiments are presented in the next section together with the analysis of the performance of the genetic algorithm.

4. COMPUTATIONAL EXPERIMENTS

In order to verify the variation of the measure (15) with the size of the problem the values of $i_{\max}$ and $t_{\max}$ were fixed at 100 and 1000, respectively.

The other parameters (p_{cross} , p_{mut}) and the proportion $E:I$ were determined after a series of experiments which make use of the statistical tests of Friedman and Wilcoxon to determine the best set of values for the parameters (Xu *et al.*, 1996). The best performance of the genetic algorithm was observed with $p_{\text{cross}} = 0.6$, $p_{\text{mut}} = 0.45$ and $E:I = 1:1$. Note that the mutation probability is higher than the traditional values found in the literature (Goldberg, 1989). This indicates that the diversifying aspect of mutation is needed to improve the performance of the genetic algorithm for the PST. In this case mutation can not be considered of secondary importance.

After the parameter determination, each one of the 750 test problems was solved 10 times. Each time, a different seed for the random number generator was used. By running the genetic algorithm with different seeds, an average measure of performance can be

obtained since the algorithm is strongly based on random numbers. Table 1 presents the average Gain computed for the 25 problem sizes. These results can be seen in Figure 4.

The heuristic presented in section 3 is very simple and it is expected that as the problem size increases its performance becomes poor. Thus, for larger problems the performance of the genetic algorithm is supposed to improve. The results of Table 1 and Figure 4 corroborate this conjecture and can be considered very good.

Table 1 - Gain (%) for the genetic algorithm with respect to the heuristic.

	n = 30	n = 50	n = 70	n = 90	n = 110
m = 2	42.832	47.775	53.436	55.014	56.568
m = 3	36.159	41.774	46.580	48.578	46.490
m = 4	27.196	39.755	38.844	42.515	43.611
m = 5	20.047	29.499	38.084	39.057	38.876
m = 6	18.295	29.135	34.352	36.001	37.540

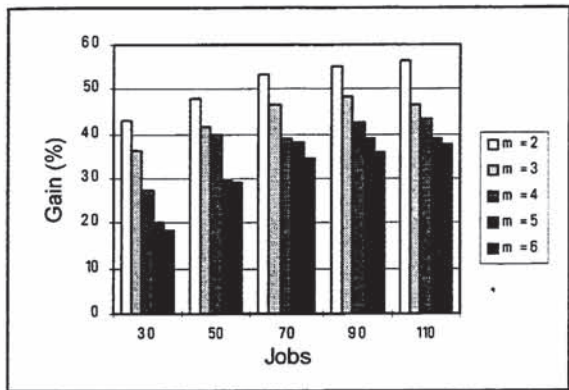


Figure 4 - Gain (%) for the 25 problem sizes.

Since the maximum number of iterations is fixed, the algorithm complexity depends only on the size of the problem. The average computational time spent by the genetic algorithm is presented in Table 2. These results indicate that with all parameters fixed, the complexity of the algorithm is almost linear with respect to m and n. Note that the time consumed for the larger problem (110 jobs and 6 machines) is less than 1 minute. This is very satisfactory and shows that the algorithm is applicable to even larger problems without loss of computational efficiency.

Table 2 - Time (seconds) spent by the genetic algorithm.

	n = 30	n = 50	n = 70	n = 90	n = 110
m = 2	18.097	26.293	34.112	43.003	52.110
m = 3	18.386	26.622	35.259	43.909	52.549
m = 4	18.810	26.945	35.550	43.938	52.630
m = 5	19.104	27.454	35.930	44.612	53.097
m = 6	19.541	27.654	36.082	45.124	53.884

Although the termination criterion of the algorithm is the maximum number of iterations, the best solution S* can be found earlier. Figure 5 shows that for small problems S* is found in a few iterations. As the problem size increases the algorithm spends more iterations improving the solutions. This can be explained by the fact that populations with 100 strings become less representative of the search space as m and n increase.

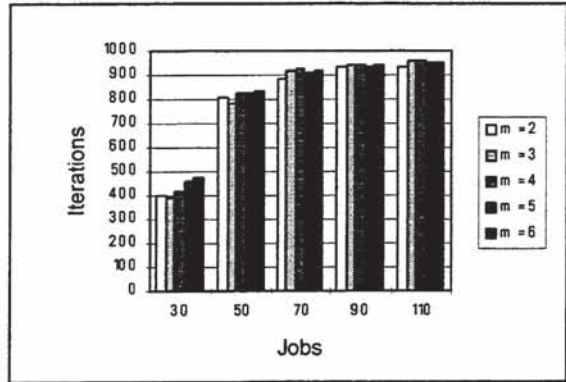


Figure 5 - Iterations to find S* for 25 problem sizes.

5. CONCLUDING REMARKS

A genetic algorithm to obtain good solutions for the problem of scheduling jobs on identical parallel machines with sequence dependent setup times for minimizing the total tardiness with respect to due dates was presented in this work.

The representation chosen for the solutions made it possible to use genetic operators which can handle the two main aspects of the problem: assignment of jobs to machines and sequencing of jobs in each machine.

The computational results show that the algorithm presented here is very efficient for the range of jobs and machines tested, solving problems with 110 jobs and 6 machines in less than 1 minute.

The tardiness values obtained were compared with the results of a simple heuristic and show that the algorithm has a good performance. This indicates that the use of genetic algorithms for solving the PST is promising. It would be interesting to investigate other versions of the genetic approach for this problem.

REFERENCES

Barr, R. S., B. L. Golden, J. P. Kelly, M. Resende and W. R. Stewart (1995). Designing and Reporting on Computational Experiments with Heuristic Methods, *Journal of Heuristics*, 1, pp. 9-32.

- Bernardo, J. J. and K.-S. Lin, (1994). An Interactive Procedure for Bi-Criteria Production Scheduling, *Computers & Operations Research*, **21**, pp. 677-688.
- Cheng, T. C. E. and C. C. S. Sin (1990). A state-of-the-art review of parallel-machine scheduling research, *European Journal of Operations Research*, **47**, pp. 271-292.
- Du, J. and J. Y.-T. Leung (1990). Minimizing Total Tardiness on One Machine is NP-hard, *Mathematics of Operations Research*, **15**, pp. 483-495.
- França, P. M., M. Gendreau, G. Laporte and F. Müller (1995). The m-Traveling Salesman Problem with Minmax Objective, *Transportation Science*, **29**, pp. 267-275.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, Reading.
- Guinet A. (1991). Textile Production Systems: a Succession of Non-identical parallel processor shops, *Journal of Operational Research Society*, **42**, pp. 655-671.
- Guinet, A. (1993). Scheduling Sequence-Dependent Jobs on Identical Parallel Machines to Minimize Completion Time Criteria, *International Journal of Production Research*, **31**, pp. 1579-1594.
- Guinet, A. (1995). Scheduling Independent Jobs on Uniform Parallel Machines to Minimize Tardiness Criteria, *Journal of Intelligent Manufacturing*, **6**, pp. 95-103.
- Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*, MIT Press/ Bradford Books, Cambridge.
- Kawaguchi, T. and S. Kyan (1988). Deterministic Scheduling in Computer Systems: A Survey, *Journal of Operations Research Society of Japan*, **31**, pp. 190-216.
- Lee, Y. H. and M. Pinedo (1992). Scheduling Jobs on Parallel Machines with Sequence Dependent Setup Times, Technical Report, Columbia University, in Lee, Y. H. and S. Kim (1993). Neural Network Application for Scheduling Jobs on Parallel Machines, *Computers & Industrial Engineering*, **25**, pp. 227-230.
- Lee, Y. H. and S. Kim (1993). Neural Network Application for Scheduling Jobs on Parallel Machines, *Computers & Industrial Engineering*, **25**, pp. 227-230.
- Michalewicz, Z. (1992). *Genetic Algorithms + Data Structures = Evolution Programs*, Springer-Verlag, Berlin.
- Rajgopal, J. and B. Bidanda (1991). On scheduling parallel machines with two setup classes, *International Journal of Production Research*, **29**, pp. 2443-2458.
- Xu, J., S. Y. Chiu and F. Glover (1996). Fine-Tuning a Tabu Search Algorithm with Statistical Tests, Working Paper, University of Colorado.